DAYANANDA SAGAR ACADEMY OF TECHNOLOGY AND MANAGEMENT

Accredited by NAAC with Grade A+

Autonomous under VTU, Belagavi

Web Technologies [23ICBB36]

Project Report

on

"Krypt"

Submitted by

TEJOMAI V [1DT23IC058]

DAKSHIJA P[1DT23IC025]

PHANEENDRA M [1DT23IC026]

GOWRISH P [1DT23IC027]

In partial fulfilment of the requirement for the degree of

BACHELOR OF ENGINEERING

DAYANANDA SAGAR ACADEMY OF TECHNOLOGY AND MANAGEMENT

Accredited by NAAC with Grade A+

Udayapura, Kanakapura Road, Bengaluru-560082

Academic Year 2024-2025

DAYANANDA SAGAR ACADEMY OF TECHNOLOGY AND MANAGEMENT

Accredited by NAAC with Grade A+

Udayapura, Kanakapura Road, Bengaluru-560082

Certificate

This is to certify that the Web Technologies Course project work entitled "**KRYPT**" carried out by **TEJOMAI V [1DT23IC058],DAKSHIJA P [1DT23IC025], PHANEENDRA M [1DT23IC026], GOWRISH P [1DT23IC027],** a Bonafide students of **DAYANANDA SAGAR ACADEMY OF TECHNOLOGY ANDMANAGEMENT** in Bachelor of Engineering of Visvesvaraya Technological University, Belagavi during the academic year 2024-2025. It is certified that all the corrections/suggestions indicated for internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirement in respect of the Web Technologies Course prescribed for the degree

Signature of Coordinators	**Signature of Guide**	**Signature of the Principal**
		Dr. Ravishankar M

External Viva **Name of the Examiner** **Signature with date**

1.

ACKNOWLEDGEMENT

I consider it a privilege to express through the pages of this report, a few words of gratitude to all those distinguished personalities who guided and inspired me in the completion of the PBL Project.

I would like to thank **Management**, DSATM, Bengaluru for providing excellent academic environment in college.

I would like to thank **Dr. Ravishankar M.**, Principal, DSATM, Bengaluru for having extended his support and encouragement during the course of work.

I would like to thank **Dr. Madhumala R B.**, Professor, Head of the department Dept. of CSE-ICB, DSATM, Bengaluru for her constant encouragement towards the completion of PBL Project.

I would like to thank our coordinator **Mr. Harish K** Assistant Professor, department of CS-IoT, Cyber Security including Blockchain, for being the guiding force towards successful completion of the PBL Project.

NAME OF THE STUDENT

TEJOMAI V - [1DT23IC058]

DAKSHIJA P - [1DT23IC025]

PHANEENDRA M - [1DT23IC026]

GOWRISH P - [1DT23IC027]

ABSTRACT

This Blockchain technology offers inherent properties of transparency, immutability, and decentralization, making it suitable for secure digital payments and auditable record keeping. However, in practical usage, Ethereum-based payment workflows remain complex for everyday users due to fragmented wallet interactions, multi-step transaction confirmations, limited human-readable feedback, and insufficient contextual information associated with transactions. This project presents **Krypt** , a lightweight Web3 decentralized application designed to simplify Ethereum transactions while preserving on-chain auditability through structured metadata storage.

The proposed system enables users to connect a browser-based wallet, initiate Ether (ETH) transfers, and associate each transaction with concise contextual metadata such as sender address, receiver address, transaction amount, timestamp, message, and keyword. A minimal Solidity smart contract is employed to persist this metadata on-chain, ensuring immutability and traceability without incurring excessive gas costs. The frontend is implemented as a single-page application using React and Vite, with Tailwind CSS for responsive user interface design. Wallet integration and blockchain communication are handled using MetaMask and ethers.js, allowing secure transaction signing without exposing private keys to the application layer.

The system architecture follows a clear separation of concerns, with smart contract logic responsible solely for data persistence and retrieval, while the client-side context provider manages wallet connectivity, transaction submission, and state synchronization. Transaction records are presented to users in a chronological feed with timestamps and links to blockchain explorers, improving transparency and user confidence. Functional testing demonstrates reliable wallet connection, successful transaction execution, and immediate reflection of on-chain data in the user interface.

Krypt serves as a proof-of-concept illustrating how minimal on-chain storage combined with a responsive frontend can significantly enhance usability and transparency in Web3 payment workflows. The design establishes a foundation for future extensions, including off-chain metadata storage, event-driven updates, support for additional wallets, and token-based transfers, making it suitable for academic exploration and further development in decentralized application design.

TABLE OF CONTENTS

ACKNOWLEDGEMENT

ABSTRACT

TABLE OF CONTENTS

LIST OF FIGURES

Chapter No.	Contents		Page No
1		Introduction	1
2		Literature Survey	3-4
	2.1	Filtering and Sorting Mechanisms	3
	2.2	User-Centered Design in Catalogs	3
	2.3	Ad-Free Websites for User Experience	3
	2.4	Role of Background Videos in Engagement	4
	2.5	Importance of Visual Consistency	4
	2.6	Trust-Building through Transparency	4
3		Proposed Method	5-8
	3.1	Catalog Organization and Structure	5
	3.2	Advanced Sorting and Filtering Mechanisms	6
	3.3	Interactive and Engaging Design	6
	3.4	User-Centered Design	7
	3.5	Ad-Free Environment	7
	3.6	Emphasis on User-Centered Design	7
	3.7	Scalability and Future Enhancements	8
4		Analysis And Design	9-10
	4.1	User Needs Analysis	9
	4.2	Prototyping	9
	4.3	Design Principles	9
	4.4	Structural Design	10
5		Implementation	11-13

	5.1	Technology Stack	11
	5.2	Project Phases	11
	5.3	Core Features Implementation	12
	5.4	Performance Enhancements	13
6		Results	14
7		Conclusion and Future scope	15-16
8		References	17

9		Appendices	18-67
	9.1	Home page code	18-21
	9.2	Transaction page code	21-45
	9.3	Index.css code	46-58
	9.4	Transation code	58-60
	9.5	Transaction code for smart contract	60-67

LIST OF FIGURES

Figure No.	Name of the figure	Page No
1.1	Krypt application logo	1
3.1	Interactive bike card	5
3.2	Filter option	6
7.1	Background video in home page	7
7.2	Transaction History	

CHAPTER 1

INTRODUCTION

Blockchain-based payment systems represent a significant advancement in digital finance by enabling decentralized, transparent, and tamper-resistant transactions. Ethereum, as one of the most widely adopted blockchain platforms, supports secure value transfer through smart contracts and decentralized applications (DApps). However, despite its technical strengths, interacting with Ethereum-based systems remains complex for many users due to fragmented wallet workflows, limited contextual information, and reliance on external blockchain explorers for transaction verification. The purpose of this project is to design and implement **Krypt**, a lightweight and user-friendly Web3 application that simplifies Ethereum transactions while preserving transparency and auditability.

Krypt is designed as a browser-based decentralized application that allows users to connect their digital wallets, send Ether (ETH), and associate each transaction with meaningful metadata such as messages and keywords. The application serves as a practical platform for demonstrating how minimal smart contract logic can be combined with a modern frontend to improve usability without compromising blockchain security principles. By presenting transaction details in a human-readable format, the system enhances user trust and understanding of on-chain activities.

Fig 1.1: Krypt application logo

The project focuses on creating a clean and responsive user interface that enables seamless wallet connection, transaction submission, and transaction history visualization. Users can view past transactions along with timestamps and links to blockchain explorers, allowing easy verification of on-chain records. The design emphasizes simplicity, clarity, and transparency, making blockchain interactions more accessible to users with varying levels of technical expertise.

CHAPTER 2

Literature survey

Blockchain-based applications have gained significant attention due to their ability to provide decentralization, transparency, and immutability in digital transactions. Ethereum, in particular, has emerged as a dominant platform for decentralized applications (DApps), enabling programmable transactions through smart contracts. Numerous research efforts and practical implementations focus on improving blockchain usability, transaction transparency, and secure wallet integration. Existing solutions range from simple wallet interfaces to complex decentralized finance (DeFi) platforms, each addressing different aspects of blockchain interaction.

2.1 Overview of Supercars Websites

Blockchain transaction applications are designed to facilitate the transfer of digital assets while maintaining a transparent and verifiable record on a distributed ledger. These applications typically rely on browser-based wallets such as MetaMask to manage user authentication and transaction signing. Prominent examples include wallet-centric platforms like MetaMask itself, payment-focused DApps, and DeFi applications that extend transaction functionality with advanced financial logic. While these platforms demonstrate the power of blockchain technology, many are optimized for technically experienced users rather than general audiences.

2.2 Strengths of Existing Platforms

Existing blockchain transaction platforms offer strong security guarantees and trustless execution through cryptographic mechanisms and smart contracts. Wallet integrations ensure that private keys remain under user control, reducing the risk of centralized breaches. Many applications also provide direct access to blockchain explorers, allowing users to verify transaction details independently. Advanced platforms further support token transfers, staking, and decentralized exchanges, showcasing the flexibility of Ethereum-based systems.

2.3 Weaknesses and Limitations

Despite their technical robustness, many blockchain applications suffer from usability challenges. Transaction workflows often involve multiple steps, including wallet connection, gas fee estimation, and confirmation prompts, which can be confusing for non-technical users. Additionally, transaction context—such as the purpose of a payment or related metadata—is rarely stored in a structured and discoverable manner on-chain. Users are frequently required to rely on external blockchain explorers to interpret raw transaction data, reducing accessibility and user confidence.

2.4 Key Features of Successful Websites

Successful blockchain applications emphasize simplicity, transparency, and secure user interaction. Intuitive user interfaces, clear transaction feedback, and minimal cognitive load are critical for improving adoption. Lightweight smart contracts that store only essential data help control gas costs while preserving auditability. Real-time transaction status indicators and human-readable transaction histories further enhance user trust and system usability.

2.5 Technologies Used in Supercars Websites

Modern blockchain applications typically employ Solidity for smart contract development and frameworks such as Hardhat or Truffle for compilation and deployment. Frontend development often uses JavaScript frameworks like React to build responsive single-page applications. Libraries such as ethers.js or web3.js facilitate communication between the frontend and the blockchain. Wallet providers like MetaMask enable secure transaction signing, while local blockchain nodes or test networks support development and testing.

2.6 Gaps in Current Solutions

Although existing platforms demonstrate strong security and decentralization, gaps remain in usability and contextual transparency. Many applications lack mechanisms to associate meaningful metadata with transactions, limiting their usefulness for auditing and human interpretation. Event-driven UI updates, simplified workflows, and better integration of on-chain.

CHAPTER 3

Proposed Method

The proposed Krypt application is designed using a systematic and modular approach that combines blockchain technology with modern web development practices. The primary objective of this project is to develop a lightweight, transparent, and user-friendly decentralized application that simplifies Ethereum transactions while enabling the storage of meaningful transaction metadata on-chain. The proposed system addresses the usability and contextual limitations identified in existing blockchain-based transaction platforms.

3.1 Platform Design and User Interface

The user interface of the Krypt application will emphasize simplicity, clarity, and responsiveness. A clean and minimal design will be adopted to reduce cognitive load and guide users through the transaction workflow efficiently. The interface will provide clear options for wallet connection, transaction input, and transaction history viewing. Responsive web design principles will ensure consistent performance across desktops, tablets, and mobile devices. Visual indicators such as loaders and status messages will be included to inform users of transaction progress and confirmation states.

3.2 Interactive Features

The application will support an intuitive transaction workflow that minimizes manual steps while maintaining security:

Wallet Connection: Users will be able to connect their browser-based wallet through a single interaction, enabling account detection and authorization.

ETH Transfer Form: A structured form will allow users to enter the recipient address, transaction amount, and optional metadata such as message and keyword.

On-Chain Recording: After transaction approval, metadata will be stored on-chain via a smart contract, ensuring transparency and immutability.

Transaction Feed: Completed transactions will be displayed in a chronological feed with timestamps and explorer links for verification.

3.3 Content Management

transaction record will include sender address, receiver address, amount, timestamp, message, and keyword. This structured data model improves human readability and auditability without exposing sensitive information or increasing gas costs unnecessarily. The frontend will present this data in a user-friendly format, enabling users to understand transaction context without relying solely on raw blockchain explorers.

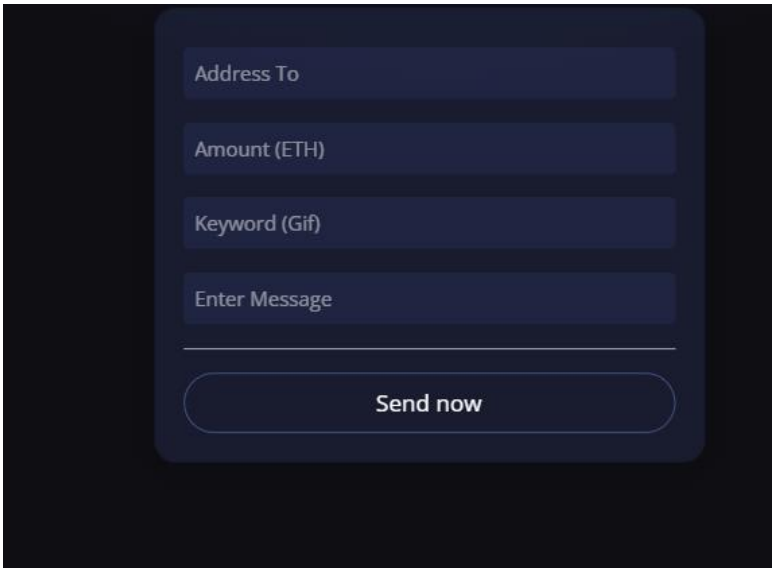

Fig3.1 : Interactive card.

3.4 Technology Stack

A modern and reliable technology stack will be employed to ensure performance and scalability:

- Smart Contract: Solidity for defining transaction metadata storage logic, deployed using the Hardhat framework.
- Frontend: React with Vite for building a fast and responsive single-page application.
- Styling: Tailwind CSS for consistent and efficient UI styling.
- Blockchain Integration: ethers.js for provider, signer, and contract interactions.

- Wallet Integration: MetaMask via window.ethereum for secure transaction signing.
- Development Network: Local Hardhat node for rapid testing and debugging.

3.5 Security and User Trust

Security is a central aspect of the proposed work. Private keys will never be handled by the application, as all signing operations are delegated to the user's wallet. Explicit wallet prompts ensure informed user consent for every transaction. The immutable nature of blockchain storage guarantees that recorded metadata remains tamper-proof and auditable, thereby enhancing user trust.

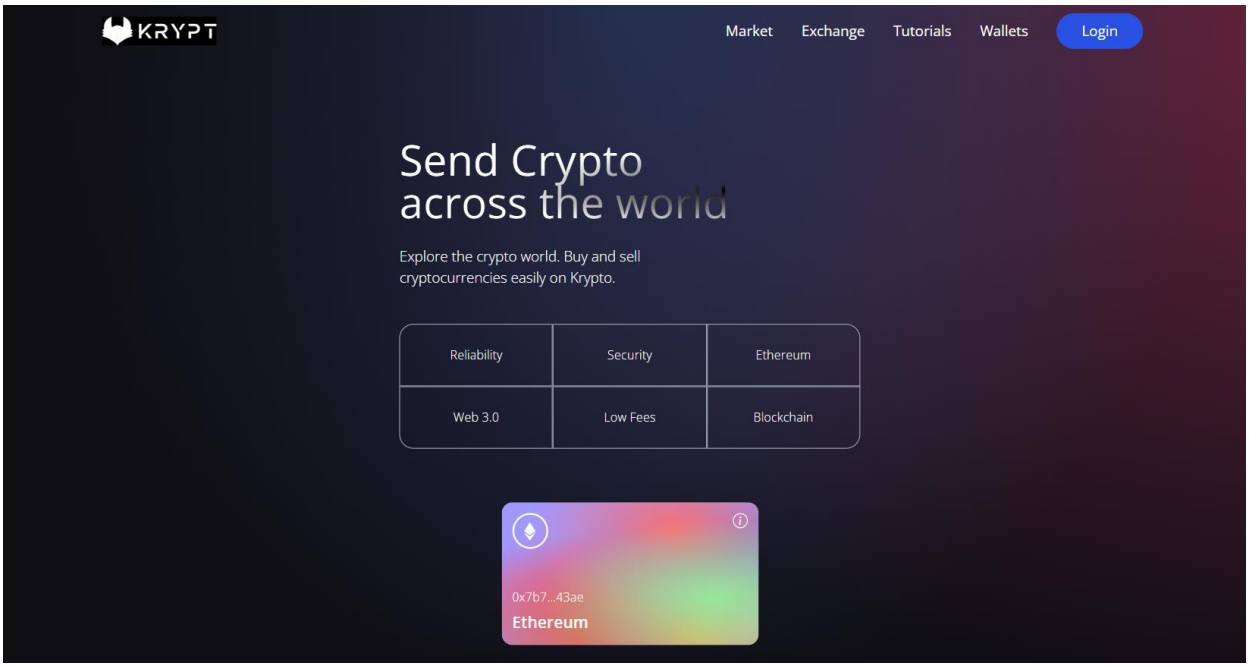

Fig 3.2 : Landing page of the website.

3.6 Blockchain for Security

The proposed architecture is designed to support future enhancements. Potential extensions include event-driven UI updates using smart contract events, integration of off-chain storage solutions such as IPFS for large metadata, support for additional wallets, and token-based transfers. These enhancements will allow the system to evolve from a proof-of-concept into a more comprehensive Web3 transaction platform.

CHAPTER 4

Analysis and Design

The analysis and design phase plays a critical role in ensuring that the Krypt application is usable, scalable, and secure. This phase focuses on identifying user requirements, defining functional and non-functional system needs, and designing an architecture that efficiently integrates blockchain technology with a modern web-based user interface. The objective is to provide a seamless Ethereum transaction experience while maintaining transparency, auditability, and security.

4.1 Requirements Analysis

The initial step in the analysis process involves identifying the functional and non-functional requirements of the proposed system. These requirements are derived from user expectations and the limitations observed in existing blockchain transaction platforms.

Functional Requirements:

- **Wallet Connection:** The system shall allow users to connect a browser-based Ethereum wallet securely.
- **ETH Transfer:** Users shall be able to send Ether to a specified recipient address.
- **Transaction Metadata Input:** Users shall be able to attach optional metadata such as a message and keyword to each transaction.
- **On-Chain Metadata Storage:** The system shall store structured transaction metadata on the blockchain using a smart contract.
- **Transaction History Display:** Users shall be able to view a list of past transactions with timestamps and explorer links.

Non-Functional Requirements:

- **Security:** Private keys must never be exposed to the application; all signing operations must occur within the wallet.
- **Usability:** The user interface should be simple, intuitive, and suitable for non-technical users.

- **Scalability:** The system should support future enhancements such as additional wallets or token transfers.
- **Performance:** The application should provide responsive feedback during transaction processing.

4.2 System Architecture Design

The system architecture defines how the various components of the application interact with each other. The proposed architecture follows a layered design consisting of frontend, blockchain layer, and development infrastructure.

Frontend (Client-Side):
The frontend is developed using React with Vite, enabling a responsive single-page application. The component-based structure supports modular development and efficient state management. The user interface handles wallet connection, transaction input, and transaction history display.

Blockchain Layer:
The blockchain layer consists of a Solidity smart contract deployed on the Ethereum network. The smart contract is responsible for storing transaction metadata and providing read functions to retrieve stored records. Blockchain interactions are facilitated using ethers.js and MetaMask.

Development and Testing Environment:
Hardhat is used for compiling, deploying, and testing the smart contract. A local Hardhat node enables rapid development and debugging before deployment to a public test network.

4.4 Data Flow Design

The data flow design illustrates how data moves through the system during user interactions.

- **Wallet Connection:** User initiates wallet connection; account information is retrieved from the wallet provider.
- **Transaction Submission:** User inputs transaction details; the frontend sends a transaction request to the wallet.

- **Blockchain Interaction:** After approval, ETH is transferred and metadata is recorded on-chain via the smart contract.
- **Data Retrieval:** Stored transaction data is fetched from the blockchain and displayed in the frontend.
- **State Update:** The frontend updates its local state to reflect newly recorded transactions.

4.5 structural design

The structural design of the Krypt application focuses on modularity, scalability, and maintainability. The system follows a layered architecture where the frontend handles presentation and user interaction, the blockchain layer manages immutable data storage, and the development framework supports deployment and testing. Clear separation of concerns ensures that changes to the user interface or smart contract logic can be made independently. This design approach enables efficient development, easier debugging, and future extensibility of the application.

CHAPTER 5

Implementation

The implementation phase focuses on transforming the proposed design and system architecture of Krypt (ChainSend) into a fully functional decentralized application. This phase involves the development of the frontend interface, smart contract implementation, wallet integration, blockchain interaction, and security enforcement. The implementation follows a structured approach to ensure usability, performance, and security while maintaining blockchain transparency.

5.1 Frontend Development

a. Setting up the Project Structure:

- Initialize the frontend project using Vite with React configuration.
- Organize the project into directories such as components, context, utilities, and assets to ensure maintainability.
- Configure Tailwind CSS for consistent styling and responsive design.

b. Implementing the User Interface:

- Develop the homepage to display wallet connection status and the transaction input form.
- Implement a transaction form that allows users to enter recipient address, ETH amount, message, and keyword.
- Design reusable UI components such as navigation bar, loader, footer, and transaction cards.
- Ensure responsive layout compatibility across desktop and mobile devices.

c. State Management:

- Implement a context provider (TransactionContext) to manage wallet connection state, transaction data, and blockchain interactions.
- Use React hooks to update UI state dynamically based on transaction status and user actions.

5.2 Smart contract Development

The blockchain logic of the application is implemented using Solidity and deployed using the Hardhat development framework.

a. Smart Contract Design:

- Develop a Solidity smart contract to store structured transaction metadata including sender, receiver, amount, timestamp, message, and keyword.
- Implement functions to add new transaction records and retrieve stored transactions.
- Maintain a transaction count variable to optimize data retrieval.

b. Contract Deployment:

- Compile and deploy the smart contract using Hardhat scripts.
- Store the generated contract address and ABI in the frontend configuration files for interaction.
- Verify correct deployment by inspecting compiled artifacts.

5.3 Interactive Features

a. Transaction History Visualization:

- Implement a transaction feed displaying sender and receiver addresses, amount, message, keyword, and timestamp.
- Provide clickable links to blockchain explorers for transaction verification.

b. User Feedback Mechanisms:

- Display loaders and status indicators during wallet connection and transaction confirmation.
- Show success or failure notifications based on transaction outcomes.

5.4 Security Implementation

a. Wallet-Based Security:

- Ensure that private keys are never accessed or stored by the application.

- All transaction signing is performed securely within the user's wallet.

b. Data Integrity and Transparency:

- Store only non-sensitive and essential metadata on-chain to prevent privacy exposure.
- Rely on blockchain immutability to ensure tamper-proof transaction records.

c. Secure Communication:

- Use wallet prompts to obtain explicit user consent for each transaction.
- Validate user inputs to prevent malformed transactions and runtime errors.

CHAPTER 6

Results

The results section presents the outcomes achieved after the successful development, integration, and testing of the Krypt decentralized application. This section evaluates the functionality of the implemented features, system performance, security effectiveness, and observations from testing. The results are analyzed in relation to the objectives defined during the project proposal phase.

6.1 Functional Features

The Krypt application has been successfully implemented with all core functional requirements operational. The system provides a simple, transparent, and secure workflow for Ethereum transactions. The major functional features achieved are as follows:

- **Wallet Connection:** Users are able to connect their browser-based Ethereum wallet seamlessly through a single interaction. The application correctly detects the active account and displays the wallet address upon successful connection.
- **ETH Transfer with Metadata:** The application allows users to send Ether by specifying the recipient address and amount, along with optional message and keyword metadata.
- **On-Chain Metadata Storage:** Transaction metadata is recorded on the blockchain using a deployed smart contract, ensuring immutability and auditability.
- **Transaction History Display:** Completed transactions are displayed in a chronological feed showing sender, receiver, amount, message, keyword, timestamp, and blockchain explorer links.
- **User Feedback Indicators:** Loading indicators and status messages provide clear feedback during transaction processing and confirmation.

6.2 Performance and Optimization

- **Transaction Processing:** Transactions are submitted efficiently through the connected wallet, with confirmation times consistent with the underlying Ethereum network conditions.

- **Frontend Performance:** The application loads quickly due to the lightweight React + Vite setup. UI components render efficiently, even when displaying multiple transaction records.
- **Responsiveness:** The interface is fully responsive and performs consistently across desktop and mobile browsers. Tailwind CSS ensures adaptive layouts for varying screen sizes.
- **State Optimization:** Local caching of transaction count reduces redundant blockchain read operations, improving performance during repeated interactions.

6.3 Security and Privacy

- **Wallet-Based Authentication:** The application does not store or handle private keys. All signing operations occur within the user's wallet, ensuring strong security.
- **Blockchain Integrity:** Transaction metadata stored on-chain remains immutable and verifiable through blockchain explorers, preventing unauthorized modification.
- **User Consent:** Every transaction requires explicit wallet confirmation, ensuring that users retain full control over transaction execution.
- **Input Validation:** User inputs are validated to prevent malformed transactions and runtime errors.

6.4 User Feedback and Testing Results

- **Functional Testing:** Manual testing confirmed successful wallet connection, ETH transfer, metadata recording, and transaction retrieval across multiple test runs.
- **Usability Feedback:** Users reported that the application is easy to use and that the transaction workflow is clear and intuitive, even for users new to blockchain technology.
- **Transparency Feedback:** The presence of human-readable transaction records and direct links to blockchain explorers improved user trust and confidence in the system.
- **Performance Feedback:** No major performance issues were observed during testing, and the application maintained stable behavior under normal usage conditions.

Overall, the results demonstrate that Krypt effectively meets its design objectives by providing a lightweight, secure, and transparent Ethereum transaction platform with improved usability and contextual clarity.

CHAPTER 7

Conclusion andFuture scope

The Krypt project has been successfully designed and implemented, achieving the primary objectives established at the outset of the development process. By integrating blockchain technology with modern web development frameworks such as React, Vite, and Solidity, the application provides a lightweight, secure, and user-friendly decentralized platform for Ethereum transactions. The system enables users to connect their digital wallets, transfer Ether, and record structured transaction metadata on-chain, thereby enhancing transparency, auditability, and usability. Through its minimal smart contract design and responsive frontend, Krypt effectively bridges the gap between complex blockchain infrastructure and accessible user interaction.

The application demonstrates reliable performance and security by leveraging wallet-based authentication, ensuring that private keys remain under user control at all times. The inclusion of human-readable transaction history, along with direct links to blockchain explorers, improves user confidence and understanding of on-chain activities. Testing and user observations indicate that the transaction workflow is intuitive, responsive, and suitable for users with varying levels of technical expertise. Overall, the project validates the effectiveness of combining thin on-chain storage with a clean client architecture to improve Ethereum transaction workflows.

In terms of future scope, the Krypt platform provides a strong foundation for further enhancements and expansion. The system can be extended to support event-driven updates using smart contract events, reducing reliance on manual data retrieval. Integration of off-chain storage solutions such as IPFS could allow richer metadata storage while minimizing gas costs and preserving privacy. Additional wallet support, including mobile wallet connectivity, can broaden accessibility and adoption. Furthermore, support for token-based transfers (ERC-20) and improved network awareness can enhance real-world applicability. With these extensions, Krypt can evolve from a proof-of-concept into a more comprehensive and scalable Web3 transaction platform, contributing to the advancement of user-centric decentralized applications.

Future Scope

Despite the successful implementation of the current version of the Krypt application, there is considerable potential for future enhancements and expansion. As blockchain adoption continues to grow, the platform can be extended to support more advanced features that further improve usability, scalability, and real-world applicability. These enhancements would allow the system to evolve beyond a proof-of-concept into a more comprehensive Web3 transaction solution.

One important area for future development is the enhancement of transaction metadata handling. While the current system stores compact metadata on-chain, future versions could integrate hybrid on-chain and off-chain storage mechanisms. For example, detailed transaction descriptions or documents could be stored off-chain using decentralized storage solutions, while cryptographic hashes are recorded on-chain. This approach would reduce gas costs while maintaining data integrity and auditability.

Another promising direction is the integration of intelligent features to improve user experience. Artificial intelligence techniques could be used to analyze user transaction behavior and provide insights such as spending summaries, transaction categorization, or contextual recommendations. Personalized dashboards could be introduced to help users better understand their transaction history and patterns, making blockchain data more accessible to non-technical users.

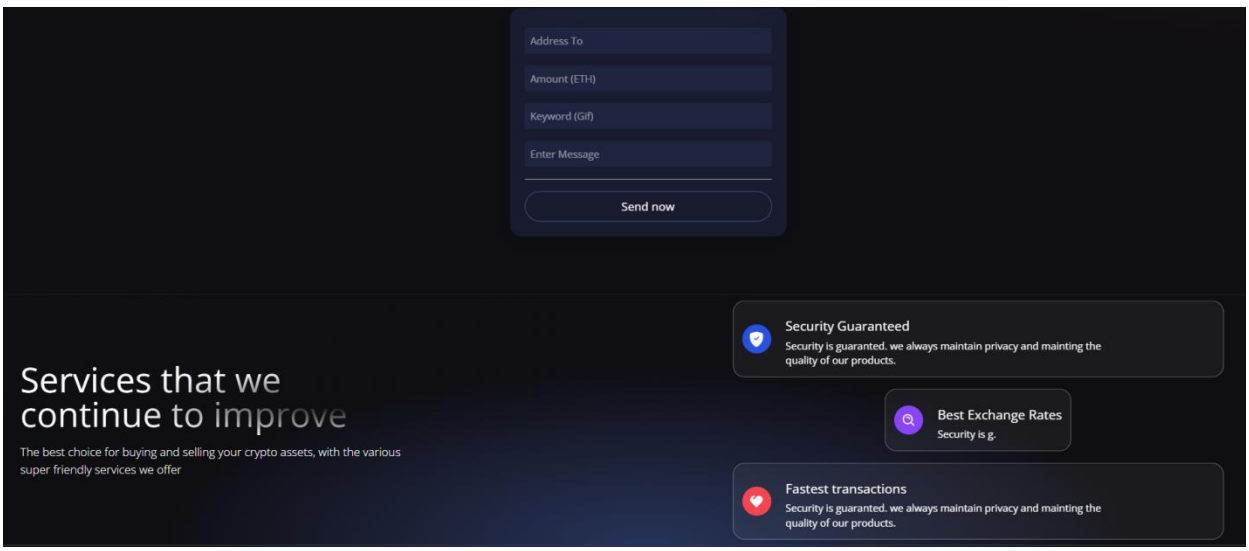

Fig 7.1: Enhanced transaction dashboard

Additionally, the platform can be extended to support a wider range of blockchain interactions. Future versions may include support for ERC-20 token transfers, multi-network compatibility, and automatic network detection. Integration with multiple wallet providers, including mobile wallets, would further increase accessibility and adoption.

Event-driven architecture represents another area of improvement. By leveraging smart contract events, the application can update transaction data in real time without relying on repeated data fetching. This would improve performance and provide a smoother user experience. Furthermore, advanced error handling and notification systems could be implemented to guide users during network mismatches or failed transactions.

Overall, the future scope of Krypt lies in expanding its functionality while preserving simplicity, security, and transparency. With these enhancements, the platform can serve as a scalable foundation for more advanced decentralized payment and transaction-tracking applications in the evolving Web3 ecosystem.

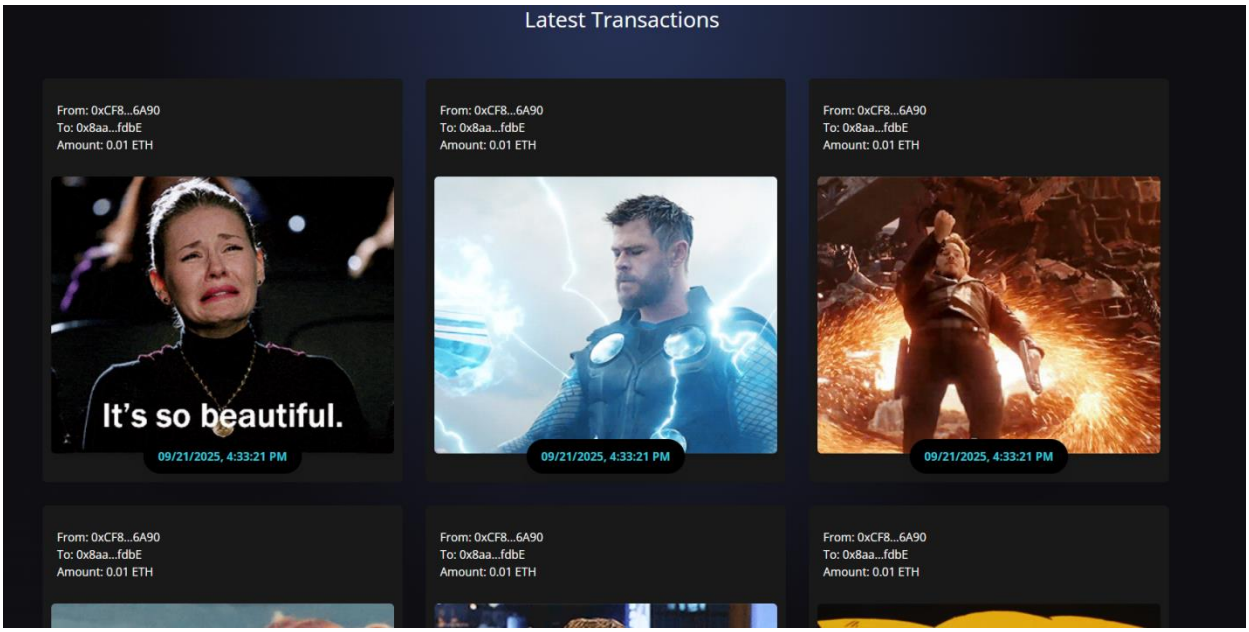

Fig 7.2: Transaction History

CHAPTER 8

References

1. React Documentation. (2024). React. https://reactjs.org/docs/getting-started.html
This reference provides detailed information on React.js, the JavaScript library used for building the user interface of the website. It helped in understanding the framework's components, states, and lifecycle methods, which were crucial for the dynamic content and interactivity of the supercars website.

2. Node.js Documentation. (2024). Node.js. https://nodejs.org/en/docs/
Node.js was used for the backend development of the website, and this documentation provided essential guidance on handling server-side processes, such as API creation, user authentication, and database interactions.

3. Blockchain Technology: A Guide for Beginners. (2023). IBM Blockchain. https://www.ibm.com/topics/what-is-blockchain
This resource helped in understanding blockchain technology and how it could be implemented for securing transactions and providing transparency in the supercars website project.

4. Augmented Reality (AR) Development with Unity. (2024). Unity. https://unity.com/solutions/augmented-reality
This guide provided the foundation for integrating augmented reality (AR) features into the website, allowing users to interact with 3D models of supercars. It helped in understanding AR concepts and tools that could be used for virtual car displays.

Chapter 9

Appendix

9.1 Home page code:

```
import React, { useContext } from "react";
import { AiFillPlayCircle } from "react-icons/ai";
import { SiEthereum } from "react-icons/si";
import { BsInfoCircle } from "react-icons/bs";

import { TransactionContext } from "../context/TransactionContext";
import { shortenAddress } from "../utils/shortenAddress";
import { Loader } from "./";

const companyCommonStyles = "min-h-[70px] sm:px-0 px-2 sm:min-w-[120px] flex justify-
center items-center border-[0.5px] border-gray-400 text-sm font-light text-white";

const Input = ({ placeholder, name, type, value, handleChange }) => (
  <input
    placeholder={placeholder}
    type={type}
    step="0.0001"
    value={value}
    onChange={(e) => handleChange(e, name)}
    className="my-2 w-full rounded-sm p-2 outline-none bg-transparent text-white border-none
text-sm white-glassmorphism"
  />
);

const Welcome = () => {
  const { currentAccount, connectWallet, handleChange, sendTransaction, formData, isLoading
} = useContext(TransactionContext);

  const handleSubmit = (e) => {
    const { addressTo, amount, keyword, message } = formData;
```

```
    e.preventDefault();

    if (!addressTo || !amount || !keyword || !message) return;

    sendTransaction({ addressTo, amount, keyword, message });
  };

  return (
    <div className="flex w-full justify-center items-center">
      <div className="flex mf:flex-row flex-col items-start justify-between md:p-20 py-12 px-
4">
        <div className="flex flex-1 justify-start items-start flex-col mf:mr-10">
          <h1 className="text-3xl sm:text-5xl text-white text-gradient py-1">
            Send Crypto <br /> across the world
          </h1>
          <p className="text-left mt-5 text-white font-light md:w-9/12 w-11/12 text-base">
            Explore the crypto world. Buy and sell cryptocurrencies easily on Krypto.
          </p>
          {!currentAccount && (
            <button
              type="button"
              onClick={connectWallet}
              className="flex flex-row justify-center items-center my-5 bg-[#2952e3] p-3 rounded-
full cursor-pointer hover:bg-[#2546bd]"
            >
              <AiFillPlayCircle className="text-white mr-2" />
              <p className="text-white text-base font-semibold">
                Connect Wallet
              </p>
            </button>
          )}
```

```
<div className="grid sm:grid-cols-3 grid-cols-2 w-full mt-10">
  <div className={`rounded-tl-2xl ${companyCommonStyles}`}>
    Reliability
  </div>
  <div className={companyCommonStyles}>Security</div>
  <div className={`sm:rounded-tr-2xl ${companyCommonStyles}`}>
    Ethereum
  </div>
  <div className={`sm:rounded-bl-2xl ${companyCommonStyles}`}>
    Web 3.0
  </div>
  <div className={companyCommonStyles}>Low Fees</div>
  <div className={`rounded-br-2xl ${companyCommonStyles}`}>
    Blockchain
  </div>
</div>
</div>

<div className="flex flex-col flex-1 items-center justify-start w-full mf:mt-0 mt-10">
  <div className="p-3 flex justify-end items-start flex-col rounded-xl h-40 sm:w-72 w-full
my-5 eth-card .white-glassmorphism ">
    <div className="flex justify-between flex-col w-full h-full">
      <div className="flex justify-between items-start">
        <div className="w-10 h-10 rounded-full border-2 border-white flex justify-center
items-center">
          <SiEthereum fontSize={21} color="#fff" />
        </div>
        <BsInfoCircle fontSize={17} color="#fff" />
      </div>
      <div>
        <p className="text-white font-light text-sm">
          {shortenAddress(currentAccount)}
        </p>
```

```
            <p className="text-white font-semibold text-lg mt-1">
              Ethereum
            </p>
          </div>
        </div>
      </div>
      <div className="p-5 sm:w-96 w-full flex flex-col justify-start items-center blue-
glassmorphism">
        <Input placeholder="Address To" name="addressTo" type="text"
value={formData.addressTo} handleChange={handleChange} />
        <Input placeholder="Amount (ETH)" name="amount" type="number"
value={formData.amount} handleChange={handleChange} />
        <Input placeholder="Keyword (Gif)" name="keyword" type="text"
value={formData.keyword} handleChange={handleChange} />
        <Input placeholder="Enter Message" name="message" type="text"
value={formData.message} handleChange={handleChange} />

        <div className="h-[1px] w-full bg-gray-400 my-2" />

        {isLoading
          ? <Loader />
          : (
            <button
              type="button"
              onClick={handleSubmit}
              className="text-white w-full mt-2 border-[1px] p-2 border-[#3d4f7c] hover:bg-
[#3d4f7c] rounded-full cursor-pointer"
            >
              Send now
            </button>
          )}
      </div>
    </div>
```

```
      </div>
    </div>
  );
};

export default Welcome;
```

9.2 :Transaction page code:

```
import React, { useContext } from "react";

import { TransactionContext } from "../context/TransactionContext";
import useFetch from "../hooks/useFetch";
import dummyData from "../utils/dummyData";
import { shortenAddress } from "../utils/shortenAddress";


const TransactionCard = ({ addressTo, addressFrom, timestamp, message, keyword, amount, url
}) => {
    const gifUrl = useFetch({ keyword });

    return (
        <div className="bg-[#181918] m-4 flex flex-1
        2xl:min-w-[450px]
        2xl:max-w-[500px]
        sm:min-w-[270px]
        sm:max-w-[300px]
        min-w-full
        flex-col p-3 rounded-md hover:shadow-2xl
        ">
            <div className="flex flex-col items-center w-full mt-3">
                <div className="display-flex justify-start w-full mb-6 p-2">
```

```
                <a href={`https://ropsten.etherscan.io/address/${addressFrom}`} target="_blank"
rel="noopener noreferrer">
                    <p className="text-white text-base">From:
{shortenAddress(addressFrom)}</p>
                </a>
                <a href={`https://ropsten.etherscan.io/address/${addressTo}`} target="_blank"
rel="noopener noreferrer">
                    <p className="text-white text-base">To: {shortenAddress(addressTo)}</p>
                </a>
                <p className="text-white text-base">Amount: {amount} ETH</p>
                {message && (
                    <>
                        <br />
                        <p className="text-white text-base">Message: {message}</p>
                    </>
                )}
            </div>
                <img
                    src={gifUrl || url}
                    alt="nature"
                    className="w-full h-64 2xl:h-96 rounded-md shadow-lg object-cover"
                />
                <div className="bg-black p-3 px-5 w-max rounded-3xl -mt-5 shadow-2xl">
                    <p className="text-[#37c7da] font-bold">{timestamp}</p>
                </div>
            </div>
        </div>


    )
}
const Transactions = () => {
    const { transactions, currentAccount } = useContext(TransactionContext);
    return(
```

```
        <div className="flex w-full justify-center items-center 2xl:px-20 gradient-bg-
transactions">
            <div className="flex flex-col md:p-12 py-12 px-4">
                {currentAccount ? (
                    <h3 className="text-white text-3xl text-center my-2">Latest Transactions</h3>
                ): (
                    <h3 className="text-white text-3xl text-center my-2">Connect your account to see
the Latest transactions</h3>
                )}
                <div className="flex flex-wrap justify-centre items-center mt-10">
                    {[...dummyData, ...transactions].reverse().map((transaction, i) => (
<TransactionCard key={i} {...transaction} />))}
                </div>
            </div>
        </div>
    );
}
export default Transactions;
```

9.3 :index.css code:

```
@import
url("https://fonts.googleapis.com/css2?family=Open+Sans:wght@300;400;500;600;700&displa
y=swap");

* html {
  padding: 0;
  margin: 0;
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: "Open Sans", sans-serif;
```

```
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

.gradient-bg-welcome {
  background-color:#0f0e13;
  background-image:
    radial-gradient(at 0% 0%, hsla(253,16%,7%,1) 0, transparent 50%),
    radial-gradient(at 50% 0%, hsla(225,39%,30%,1) 0, transparent 50%),
    radial-gradient(at 100% 0%, hsla(339,49%,30%,1) 0, transparent 50%);
}

.gradient-bg-services {
  background-color:#0f0e13;
  background-image:
    radial-gradient(at 0% 0%, hsla(253,16%,7%,1) 0, transparent 50%),
    radial-gradient(at 50% 100%, hsla(225,39%,25%,1) 0, transparent 50%);
}

.gradient-bg-transactions {
  background-color: #0f0e13;
  background-image:
    radial-gradient(at 0% 100%, hsla(253,16%,7%,1) 0, transparent 50%),
    radial-gradient(at 50% 0%, hsla(225,39%,25%,1) 0, transparent 50%);
}

.gradient-bg-footer {
  background-color: #0f0e13;
  background-image:
    radial-gradient(at 0% 100%, hsla(253,16%,7%,1) 0, transparent 53%),
    radial-gradient(at 50% 150%, hsla(339,49%,30%,1) 0, transparent 50%);
}
```

```
.blue-glassmorphism {
  background: rgb(39, 51, 89, 0.4);
  border-radius: 16px;
  box-shadow: 0 4px 30px rgba(0, 0, 0, 0.2);
  backdrop-filter: blur(5px);
  -webkit-backdrop-filter: blur(5px);
  border: 1px solid rgba(0, 0, 0, 0.3);
}

/* white glassmorphism */
.white-glassmorphism {
  background: rgba(255, 255, 255, 0.05);
  border-radius: 16px;
  backdrop-filter: blur(5px);
  -webkit-backdrop-filter: blur(5px);
  border: 1px solid rgba(255, 255, 255, 0.3);
}

.eth-card {
  background-color:#a099ff;
  background-image:
    radial-gradient(at 83% 67%, rgb(152, 231, 156) 0, transparent 58%),
    radial-gradient(at 67% 20%, hsla(357,94%,71%,1) 0, transparent 59%),
    radial-gradient(at 88% 35%, hsla(222,81%,65%,1) 0, transparent 50%),
    radial-gradient(at 31% 91%, hsla(9,61%,61%,1) 0, transparent 52%),
    radial-gradient(at 27% 71%, hsla(336,91%,65%,1) 0, transparent 49%),
    radial-gradient(at 74% 89%, hsla(30,98%,65%,1) 0, transparent 51%),
    radial-gradient(at 53% 75%, hsla(174,94%,68%,1) 0, transparent 45%);
}

.text-gradient {
  background-color: #fff;
```

```
  background-image: radial-gradient(at 4% 36%, hsla(0,0%,100%,1) 0, transparent 53%), radial-gradient(at 100% 60%, rgb(0, 0, 0) 0, transparent 50%);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
}

@tailwind base;
@tailwind components;
@tailwind utilities;
@import "tailwindcss";
```

9.4 : App.css code:

```
#root {
  max-width: 1280px;
  margin: 0 auto;
  padding: 2rem;
  text-align: center;
}

.logo {
  height: 6em;
  padding: 1.5em;
  will-change: filter;
  transition: filter 300ms;
}
.logo:hover {
  filter: drop-shadow(0 0 2em #646cffaa);
}
.logo.react:hover {
  filter: drop-shadow(0 0 2em #61dafbaa);
}
```

```
@keyframes logo-spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}

@media (prefers-reduced-motion: no-preference) {
  a:nth-of-type(2) .logo {
    animation: logo-spin infinite 20s linear;
  }
}

.card {
  padding: 2em;
}

.read-the-docs {
  color: #888;
}
```

9.4 :Transaction Context:

```
import React, { useEffect, useState } from "react";
import { ethers } from "ethers";
import { contractABI, contractAddress } from "../utils/constants";
export const TransactionContext = React.createContext();

const getEthereum = () => window.ethereum;
```

```
const createEthereumContract = async () => {
  const ethereum = getEthereum();
  if (!ethereum) {
    console.log("Ethereum object not found when creating contract");
    return null;
  }

  // Support ethers v5 and v6 providers
  if (ethers.providers && ethers.providers.Web3Provider) {
    const provider = new ethers.providers.Web3Provider(ethereum);
    const signer = provider.getSigner();
    const transactionsContract = new ethers.Contract(contractAddress, contractABI, signer);
    return transactionsContract;
  }

  if (ethers.BrowserProvider) {
    const provider = new ethers.BrowserProvider(ethereum);
    const signer = await provider.getSigner();
    const transactionsContract = new ethers.Contract(contractAddress, contractABI, signer);
    return transactionsContract;
  }

  console.log('Unsupported ethers version');
  return null;
};

export const TransactionProvider = ({ children }) => {
  const [formData, setformData] = useState({ addressTo: "", amount: "", keyword: "", message:
"" });
  const [currentAccount, setCurrentAccount] = useState("");
  const [isLoading, setIsLoading] = useState(false);
  const [transactionCount, setTransactionCount] =
useState(localStorage.getItem("transactionCount"));
```

```
const [transactions, setTransactions] = useState([]);

const handleChange = (e, name) => {
  setformData((prevState) => ({ ...prevState, [name]: e.target.value }));
};

const getAllTransactions = async () => {
  try {
    const ethereum = getEthereum();
    if (ethereum) {
      const transactionsContract = await createEthereumContract();
      if (!transactionsContract) return;

      const availableTransactions = await transactionsContract.getAllTransactions();

      const structuredTransactions = availableTransactions.map((transaction) => ({
        addressTo: transaction.receiver,
        addressFrom: transaction.sender,
        timestamp: new Date(transaction.timestamp.toNumber() * 1000).toLocaleString(),
        message: transaction.message,
        keyword: transaction.keyword,
        amount: parseInt(transaction.amount._hex) / (10 ** 18)
      }));

      console.log(structuredTransactions);

      setTransactions(structuredTransactions);
    } else {
      console.log("Ethereum is not present");
    }
  } catch (error) {
    console.log(error);
  }
```

```
};

const checkIfWalletIsConnect = async () => {
  try {
    const ethereum = getEthereum();
    if (!ethereum) return alert("Please install MetaMask.");

    const accounts = await ethereum.request({ method: "eth_accounts" });

    if (accounts.length) {
      setCurrentAccount(accounts[0]);

      getAllTransactions();
    } else {
      console.log("No accounts found");
    }
  } catch (error) {
    console.log(error);
  }
};

const checkIfTransactionsExists = async () => {
  try {
    const ethereum = getEthereum();
    if (ethereum) {
      const transactionsContract = await createEthereumContract();
      if (!transactionsContract) return;
      const currentTransactionCount = await transactionsContract.getTransactionCount();

      window.localStorage.setItem("transactionCount", currentTransactionCount);
    }
  } catch (error) {
    console.log(error);
```

```
    // avoid throwing to prevent uncaught exceptions in UI event handlers
  }
};

const connectWallet = async () => {
  try {
    const ethereum = getEthereum();
    if (!ethereum) return alert("Please install MetaMask.");

    const accounts = await ethereum.request({ method: "eth_requestAccounts" });

    setCurrentAccount(accounts[0]);
    window.location.reload();
  } catch (error) {
    console.log(error);
    // avoid throwing to prevent uncaught exceptions in UI event handlers
  }
};

const sendTransaction = async () => {
  try {
    const ethereum = getEthereum();
    if (ethereum) {
      const { addressTo, amount, keyword, message } = formData;
      const transactionsContract = await createEthereumContract();
      const parsedAmount = ethers.utils.parseEther(amount);

      await ethereum.request({
        method: "eth_sendTransaction",
        params: [{
          from: currentAccount,
          to: addressTo,
          gas: "0x5208", // 21000 Gwei
```

```
            value: parsedAmount._hex,
          }],
        });

        // Call contract to record the transaction details on-chain (if required by your contract)
        if (transactionsContract) {
          const transactionHash = await transactionsContract.addToBlockchain(addressTo,
parsedAmount, message, keyword);

          setIsLoading(true);
          console.log(`Loading - ${transactionHash.hash}`);
          await transactionHash.wait();
          console.log(`success ${transactionHash.hash}`);
          setIsLoading(false);

          const transactionsCount = await transactionsContract.getTransactionCount();

          setTransactionCount(transactionsCount.toNumber());
          // optionally reload or update state
          window.location.reload();
        } else {
          console.log("Transactions contract not available");
        }
      } else {
        console.log("No ethereum object");
      }
    } catch (error) {
      console.log(error);
      // avoid throwing to prevent uncaught exceptions in UI event handlers
    }
  };

  useEffect(() => {
```

```
    checkIfWalletIsConnect();
    checkIfTransactionsExists();
  }, [transactionCount]);

  return (
    <TransactionContext.Provider
      value={{
        transactionCount,
        connectWallet,
        transactions,
        currentAccount,
        isLoading,
        sendTransaction,
        handleChange,
        formData,
      }}
    >
      {children}
    </TransactionContext.Provider>
  );
};
```

9.5 : Services code:

```
import React from "react";
import { BsShieldFillCheck } from "react-icons/bs";
import { BiColor, BiSearchAlt } from "react-icons/bi";
import { RiHeart2Fill } from "react-icons/ri";

const ServicesCard = ({ color, title, icon, subtitle }) => (
    <div className="flex flex-row justify-start items-center white-glassmorphism p-3 m-2
cursor-pointer hover:shadow-xl">
        <div className={`w-10 h-10 rounded-full flex justify-center items-center ${color}`}>
            {icon}
```

```
        </div>
        <div className="ml-5 flex flex-col flex-1">
            <h3 className="mt-2 text-white text-lg">{title}</h3>
            <p className="mt-1 text-white text-sm md:w-9/12">{subtitle}</p>
        </div>

    </div>
);

const Services = () => {
    return(
        <div className="flex flex-col md:flex-row w-full justify-center items-center gradient-bg-
services">
            <div className="flex mf:flex-row flex-col item-center justify-between md:p-20 py-
12px-4">
                <div className="flex-1 flex flex-col justify-start items-start">
                    <h1 className="text-white text-3xl sm:text-5xl py-2 text-gradient">
                        Services that we
                        <br />
                        continue to improve
                    </h1>
                    <p className="text-left my-2 text-white font-light md:w-9/12 w-11/12 text-base">
          The best choice for buying and selling your crypto assets, with the
          various super friendly services we offer
        </p>
                </div>
            </div>
            <div className="flex-1 flex flex-col justify-start items-center">
                <ServicesCard
                    color="bg-[#2952e3]"
                    title="Security Guaranteed"
                    icon={<BsShieldFillCheck fontSize={21} className="text-white" />}
```

```
                subtitle="Security is guaranted. we always maintain privacy and mainting the
quality of our products."
            />

            <ServicesCard
                color="bg-[#8945F8]"
                title="Best Exchange Rates"
                icon={<BiSearchAlt fontSize={21} className="text-white" />}
                subtitle="Security is g."
            />

            <ServicesCard
                color="bg-[#F84550]"
                title="Fastest transactions"
                icon={<RiHeart2Fill fontSize={21} className="text-white" />}
                subtitle="Security is guaranted. we always maintain privacy and mainting the
quality of our products."
            />
        </div>
    </div>
  );
}
export default Services;
```

9.6 : Transaction code for smart contract:

```
// SPDX-License-Identifier: UNLICENSED

pragma solidity ^0.8.0;

contract Transactions {
    uint256 transactionCount;
```

```
    event Transfer(address from, address receiver, uint amount, string message, uint256
timestamp, string keyword);

    struct TransferStruct {
        address sender;
        address receiver;
        uint amount;
        string message;
        uint256 timestamp;
        string keyword;
    }

    TransferStruct[] transactions;

    function addToBlockchain(address payable receiver, uint amount, string memory message,
string memory keyword) public {
        transactionCount += 1;
        transactions.push(TransferStruct(msg.sender, receiver, amount, message, block.timestamp,
keyword));

        emit Transfer(msg.sender, receiver, amount, message, block.timestamp, keyword);
    }

    function getAllTransactions() public view returns (TransferStruct[] memory) {
        return transactions;
    }

    function getTransactionCount() public view returns (uint256) {
        return transactionCount;
    }
}
```